

Conception — Lot 3 : Gestion des comptes par l'intendance

Document de conception — FIGÉ. Version 1.0 — 2026-07-17. On code **phase par phase** dans cet ordre (P0 → P5).

1. Objectif

Basculer la création de comptes **résidentes** d'un modèle **self-service** (chacune s'inscrit et choisit sa résidence/chambre) vers un modèle **piloté par l'intendance** :

Une **chambre = une place**. Le nombre de comptes résidentes est **plafonné par le nombre de chambres**. L'intendance **ouvre une place → invite → l'étudiante complète son compte → au départ, l'admin libère la place** (compte **archivé**, historique conservé) → la chambre est **réattribuée**.

Les **prestataires (corail)** sont gérées selon le même cycle, mais sur des **postes** (rôles) plutôt que des chambres.

2. Décisions déjà actées (réunion 2026-07-10)

#	Décision	Choix
D1	Capacité d'une chambre	1 chambre = 1 place = 1 compte
D2	Comptes « invitées » (invitees)	Inchangés : gardent le self-signup, n'occupent pas de place
D3	Départ d'une résidente	Archivage (compte désactivé, historique repas/présence conservé pour la compta)
D4	Migration de l'existant	Aucune reprise à risque : l'appli est fermée et la promo largement renouvelée → les admins recréent chambres/postes (écran dédié, cf. D9) et invitent la nouvelle promo. Les comptes actuels sont archivables en masse .
D5	Corail (prestataires)	Gérées comme des postes (places de rôle, sans chambre), même cycle de vie ; pas de plafond ; libellé de poste libre (l'admin nomme : Cuisine, Ménage...)
D6	Mécanisme d'invitation	Email (invite Supabase + lien magique) — l'envoi d'emails fonctionne déjà. Expiration : 14 jours (défaut Supabase).
D7	Profil à l'acceptation	Résidence / (étage / chambre ou poste) imposés et verrouillés ; l'étudiante ne saisit que mot de passe + infos perso
D8	Super-admin (compte de Loan)	Compte technique à accès total, hors modèle résidente : pas de place, non compté dans la capacité, exclu des listes (présences, repas, compta, sélection), non archivable/rétrogradable par les autres. Migration : bascule du compte actuel (qui occupe une chambre) vers ce statut → libère la chambre.

#	Décision	Choix
D9	Écran de gestion des chambres/postes	Obligatoire in-app : les admins créent/éditent/désactivent chambres et postes eux-mêmes (autonomie maximale, objectif « le moins besoin de Loan »).
D10	Déplacement interne	Une admin peut déplacer une résidente active d'une chambre à une autre (déménagement).

3. Modèle de données (proposition)

3.1. Table `places` (chambres et postes)

Unifie les deux types de « place » pour partager tout le cycle de vie.

Colonne	Type	Détail
<code>id</code>	uuid	PK
<code>residence</code>	text	12 36 corail
<code>kind</code>	text	chambre poste
<code>etage</code>	text null	renseigné pour les chambres, null pour les postes
<code>code</code>	text	code chambre (ex. <code>grand_palais</code>) ou libellé de poste (ex. <code>Cuisine</code>)
<code>label</code>	text null	libellé lisible affiché (sinon dérivé de <code>code</code>)
<code>is_active</code>	bool	une place désactivée n'est plus proposée (ex. chambre en travaux)
<code>created_at</code>	timestampz	

Occupation déduite, pas stockée : une place est « occupée » s'il existe un compte **actif** avec `place_id = places.id`, « libre » sinon. Évite les incohérences (une seule source de vérité).

3.2. Table `residentes` — colonnes ajoutées

Colonne	Type	Détail
<code>place_id</code>	uuid null	FK → <code>places.id</code> (null pour invitées et super-admin)
<code>statut</code>	text	active archivee (défaut <code>active</code>)
<code>archived_at</code>	timestampz null	date de libération de la place
<code>is_super_admin</code>	bool	compte technique de Loan : accès total, hors modèle résidente (cf. D8, R-CPT-08). Défaut <code>false</code> . Implique <code>is_admin</code> .

Les champs texte existants `residence` / `etage` / `chambre` sont **conservés et synchronisés** depuis la place (rétrocompatibilité : tout le code actuel les lit). À terme on pourra ne garder que `place_id`.

3.3. Table `invitations`

Suit l'état des invitations en attente (liste, relance, expiration).

Colonne	Type	Détail
<code>id</code>	uuid	PK
<code>email</code>	text	destinataire
<code>place_id</code>	uuid	FK → <code>places.id</code> (place réservée)
<code>role</code>	text	<code>residente</code> (extensible)
<code>auth_user_id</code>	uuid null	id du user Supabase créé par l'invite
<code>statut</code>	text	<code>envoyee</code> <code>acceptee</code> <code>expiree</code> <code>annulee</code>
<code>invited_by</code>	uuid	admin émettrice
<code>expires_at</code>	timestampz	ex. +14 jours
<code>created_at</code>	timestampz	

4. Parcours (flows)

4.1. Ouvrir une place & inviter

1. L'admin ouvre l'écran **Chambres / Postes**, choisit une place **libre** (ou crée un poste corail).
2. Elle saisit l'**email** de la future occupante → « **Inviter** ».
3. Côté serveur (service role) : `auth.admin.inviteUserByEmail(email, { data: { role, place_id }, redirectTo })` + création d'une ligne `invitations` (`statut=envoyee`, `expires_at`).
4. L'étudiante reçoit un **email** avec un lien.

Anti-doublon : impossible d'inviter sur une place **déjà occupée par un compte actif** ou **avec une invitation en attente**.

4.2. Accepter l'invitation

1. L'étudiante clique le lien → page « **Activer mon compte** ».
2. Elle définit son **mot de passe** et complète ses **infos perso** (nom, prénom, date de naissance...).
3. La **résidence / étage / chambre (ou poste)** sont **pré-remplis et verrouillés** (issus de `place_id`).
4. À la validation : création de la ligne `residentes` (`statut=active`, `place_id`), `invitations.statut=acceptee`. La place devient « occupée ».

4.3. Libérer une place (départ)

1. L'admin ouvre la place occupée → « **Libérer / Archiver l'occupante** » (confirmation).
2. `residentes.statut=archivee`, `archived_at=now()`, `place_id` **conservé pour l'historique** mais la place est **considérée libre** (car plus de compte *actif* dessus).

Le compte archivé : **ne peut plus se connecter, disparaît** des listes actives (présences, compta courante, sélection repas), mais ses **repas/présences passés restent** → compta des mois écoulés intacte.

4.4. Réattribuer

Une place libérée (ou un poste corail dont la prestataire est partie) est **réattribuable** immédiatement (retour en 4.1). C'est le cas du **remplacement d'une prestataire**.

4.5. Relancer / annuler une invitation

- **Relancer** : renvoie l'email (nouveau lien), repousse `expires_at` .
- **Annuler** : `invitations.statut=annulee` , la place redevient librement ré-ouvrable.
- **Expiration** : passé `expires_at` sans acceptation → `expiree` (place ré-ouvrable).

5. Cas limites & garde-fous

- **Super-admin** : compte hors modèle résidente (D8/R-CPT-08) — jamais listé côté résidentes, jamais archivable/rétrogradable par une autre admin, ne consomme pas de place.
- **Dernière administratrice** : on ne peut pas rétrograder/archiver la dernière admin active (le super-admin garde toujours la main).
- **Déplacement interne** (R-CPT-09) : « Déplacer » une résidente vers une place libre ; refusé si la place cible est occupée.
- **Chambre occupée** : invitation refusée (cf. 4.1).
- **Invitée (invitees) : hors périmètre** — parcours self-signup inchangé, pas de place.
- **Corail** : postes sans étage/chambre ; pas de plafond ; sinon cycle identique.
- **Compte archivé réactivé ?** Hors périmètre v1 (si une ancienne revient, on réinvite sur une place).
- **Migration** : voir §7.
- **Self-signup résidentes : désactivé** (l'inscription résidente passe désormais par l'invitation). Le self-signup **invitées** reste.

6. Règles (à intégrer aux specs — `R-CPT-xx`)

- **R-CPT-01** — Un compte résidente est **rattaché à exactement une place** (chambre ou poste) ; une place active porte **au plus un compte actif**.
- **R-CPT-02** — La création d'un compte résidente/corail se fait **uniquement par invitation** d'une administratrice (plus de self-signup pour ces rôles).
- **R-CPT-03** — À l'acceptation, **résidence / étage / chambre (ou poste)** sont **imposés** par l'invitation et non modifiables par l'étudiante.
- **R-CPT-04** — Le départ **archive** le compte (`statut=archivee`) : plus de connexion, retiré des vues actives, **historique conservé** pour la comptabilité.
- **R-CPT-05** — Une place est **occupée** ssi un compte **actif** y est rattaché ; sinon **libre** et réattribuable.

- **R-CPT-06** — Les **invitées** conservent le self-signup et **n'occupent aucune place**.
- **R-CPT-07** — Corail : places de type **poste** (sans étage/chambre), **sans plafond, libellé libre** ; même cycle de vie.
- **R-CPT-08** — Le **super-admin** (`is_super_admin=true`) est un **compte technique à accès total** : **sans place, non compté** dans la capacité, **exclu** des listes résidentes (présences, repas, compta, sélection), et **non archivable/rétrogradable** par les autres administratrices. (*Règle aussi le suivi Lot 2 « le compte technique apparaît dans la compta ».*)
- **R-CPT-09** — Une administratrice peut **déplacer** une résidente **active** vers une autre place libre (déménagement interne) : la place d'origine se libère, la nouvelle s'occupe.

7. Migration (D4 — sans risque)

L'appli est **fermée** et la promo **renouvelée** → **pas de reprise fine de l'existant**. On repart propre :

1. **Créer les tables** (`places` , colonnes `residentes` , `invitations`) — SQL idempotent (P0).
2. **Basculer le compte de Loan en super-admin** : `is_super_admin=true` , `place_id=null` (libère sa chambre actuelle). Script SQL ciblé.
3. **Archiver en masse** les comptes résidentes de l'ancienne promo (`statut=archivée`) — optionnel, à la convenance des admins (l'historique reste consultable).
4. Les admins **créent les chambres/postes** via l'écran dédié (D9/P1) puis **invitent** la nouvelle promo.

Un **seed de démarrage** (générer des places à partir des tuples résidence/étage/chambre existants) reste possible **en option** si ça fait gagner du temps, mais n'est **pas requis**.

8. Impact sur l'existant

- **signupForm** : le rôle « résidente » (et corail) retiré du self-signup ; ne reste que « invitée ». Nouvel écran « **Activer mon compte** » pour l'acceptation.
- **Listes actives** (présences `/admin/foyer` , compta `/admin/repas-v2` , sélection repas) : filtrer `statut=active` .
- **Panneau Administration** : nouvel onglet/écran « **Chambres & postes** ».
- Tout le code lisant `residence/etage/chambre` continue de fonctionner (champs synchronisés depuis la place).

9. Découpage en phases & estimation (~28–40 h)

Phase	Contenu	Est.
P0	Modèle de données (<code>places</code> , colonnes <code>residentes</code> dont <code>is_super_admin</code> , <code>invitations</code>) + SQL de bascule super-admin + archivage optionnel	5–7 h

Phase	Contenu	Est.
P1	Écran admin Chambres & Postes : créer / éditer / désactiver (D9) + liste par résidence/étage, occupant, statut	8–10 h
P2	Ouvrir une place + inviter (API service role, email, table invitations, relance/annulation)	7–9 h
P3	Page « Activer mon compte » (mot de passe + profil verrouillé)	5–7 h
P4	Libérer/archiver + réattribuer + cohérence compta/présences (filtrer archivées)	4–6 h
P5	Désactivation self-signup résidentes + docs (R–CPT–xx , manuels) + déploiement par blocs	3–4 h

10. Décisions §10 — tranchées (2026-07-17)

#	Question	Décision
Q1	Expiration d'invitation	✅ 14 jours (on s'aligne sur Supabase)
Q2	Libellé des postes corail	✅ Libre (l'admin nomme)
Q3	Déplacement interne d'une résidente	✅ Oui (R-CPT-09)
Q4	Écran de gestion des chambres/postes	✅ Oui, obligatoire (D9 — autonomie admins)
Q5	Statut des invitations visible/relançable	✅ Oui

Conception figée (v1.0). Implémentation P0 → P5, testée puis déployée par blocs.